RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

"CyberID – Cybersecurity Intrusion Detection System"

SOFTWARE ENGINEERING
(21IS62)
2023-2024

Submitted by

R Keerthana Prasad	**1RVU23CSE363**
N Sukhitha Bhushan	**1RVU23CSE293**
Swathi Kulkarni	**1RVU23CSE492**
Jaee Priyadarshan Kulkarni	**1RVU23CSE196**

Under the guidance of

Dr. CVSN.
Assistant Professor
Department of Computer Science and Engineering
RV College of Engineering-560059

RV COLLEGE OF ENGINEERING®, BENGALURU-59

(Autonomous Institution Affiliated to VTU, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the project work titled **CyberID – Cybersecurity Intrusion Detection System** is carried out by **R Keerthana Prasad (1RVU23CSE363), N Sukhitha Bhushan (1RVU23CSE293), Swathi Kulkarni (1RVU23CSE492), Jaee Priyadarshan Kulkarni (1RVU23CSE196)** in partial fulfillment of the completion of the course **Agile Software Engineering and Devops** with course code **CS2004** of the VI Semester Computer Science Engineering programme, during the academic year 2023-2024. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the project report and duly approved by the faculty.

Signature of Faculty **Signature of Head of the Department**

ABSTRACT

This project involves the development of an intelligent intrusion detection system designed to

identify and classify abnormal network activities using the KDDTrain dataset. Aimed at enhancing cybersecurity infrastructure, the system leverages a Support Vector Machine (SVM) with a Radial Basis Function (RBF) kernel to achieve high accuracy in distinguishing between normal and suspicious traffic patterns. By replicating the analytical capabilities of a security analyst, the system automates the detection process to improve response time and reduce the burden on IT personnel.

The solution is deployed as a Flask-based web application, allowing seamless user interaction for uploading network log files and receiving real-time classification results. In addition to threat detection, the platform integrates explainable AI components, offering transparent justifications for each prediction and fostering greater user trust. The application also features a unified dashboard that visualizes key network statistics, tracks incident patterns, and supports informed decision-making.

Overall, the project aims to modernize intrusion detection through machine learning and intelligent automation, improving the efficiency, accuracy, and interpretability of threat identification. This contributes to a more proactive and resilient cybersecurity posture for organizations facing ever-evolving digital threats.

Table of Contents

1. Introduction

1.1 Project Domain and Problem Addressing

The Intrusion Detection System (IDS) project leverages machine learning to identify potential security threats in network traffic. Using the KDDTrain dataset, we developed a robust classification system that distinguishes between normal and abnormal network activities. The system employs a Support Vector Machine (SVM) with a Radial Basis Function (RBF) kernel, chosen for its high accuracy in detecting anomalies.
To ensure reliable predictions, the data undergoes thorough preprocessing, including feature scaling using MinMaxScaler. The trained model is deployed via a Flask-based web application, allowing users to upload network logs for real-time analysis. The system provides detailed insights into detected threats, including explanations for flagged activities, making it a valuable tool for cybersecurity monitoring. By focusing on SVM, we achieved a balance between performance and interpretability, ensuring both accuracy and actionable results.

1.2 Issues and Challenges

This project addresses several key challenges in the domain of network security, particularly focusing on the detection and analysis of abnormal activities within network traffic. One of the primary issues it confronts is the difficulty in manually identifying intrusions from large volumes of network log data, which is often tedious, error-prone, and requires expert knowledge. By automating the intrusion detection process through a machine learning-based system integrated into a web application, the project enhances both the speed and reliability of threat identification, reducing the workload on cybersecurity analysts.

Another significant challenge tackled by the project is the lack of interpretability in many machine learning models used for network security. To address this, the system incorporates an explanation module that provides clear, human-readable justifications for each detection of abnormal behavior. This helps users understand why a certain log was flagged as suspicious, thereby increasing trust and transparency in the system's decisions.

Furthermore, the project solves the problem of disjointed analysis tools by offering an all-in-one dashboard that combines file upload, log analysis, threat classification, and visual analytics. This integration ensures a streamlined workflow, from data input to actionable insights, and supports continuous improvement in organizational security posture. Overall, the project aims to make intrusion detection more accessible, efficient, and explainable, ultimately contributing to stronger and more proactive network defense mechanisms.

1.3 Problem statement

Modern networks face a constant threat from a wide range of cyberattacks, making intrusion detection a critical component of any security infrastructure. However, traditional intrusion detection systems often rely on manual inspection or rule-based methods, which can be inefficient, time-consuming, and ineffective against evolving threats. Furthermore, many existing solutions lack the ability to provide clear explanations for their predictions, making it difficult for security personnel to interpret and trust automated alerts.

There is a pressing need for an intelligent, user-friendly system capable of accurately identifying abnormal network behavior while offering transparency and ease of use. This project aims to develop a machine learning-powered web application for intrusion detection that enables users to upload network traffic data, automatically classify logs as normal or abnormal, and visualize key statistical insights. By integrating explainable AI techniques and providing a comprehensive dashboard for analysis, the system seeks to enhance threat detection capabilities, reduce false positives, and support informed decision-making in cybersecurity operations.

1.4 Requirements and Feasibility Study

1. **Data Requirements:** The project requires access to a comprehensive, labeled dataset of network traffic logs, such as the KDDTrain dataset. This dataset must include a variety of normal and abnormal connection records representing different types of attacks (e.g., DoS, probe, R2L, and U2R). The availability of such datasets is feasible, as benchmark datasets like KDDCup 99 and NSL-KDD are publicly accessible and widely used in intrusion detection research.

2. **Technological Requirements:** The system is built upon machine learning algorithms—specifically a Support Vector Machine (SVM) with an RBF kernel—selected for its strong performance in binary classification tasks. Python libraries such as Scikit-learn and Flask support the development of both the backend model and the web interface. Minimal hardware requirements make this project feasible for deployment on local servers or lightweight cloud environments. Additional preprocessing tools, such as `MinMaxScaler`, ensure the model performs consistently across various input formats.

3. **Web Deployment:**A lightweight web application is required to allow users to upload log files and receive real-time analysis. This is implemented using Flask for the backend, with HTML, CSS, and JavaScript handling the frontend. The application is hosted on GitHub, offering a version-controlled, accessible, and cost-effective deployment solution. This setup supports easy updates and collaboration while keeping infrastructure overhead minimal.

4. **Visualization Tools:** The project benefits from visual representations of network traffic and detection results. Libraries such as Matplotlib, Seaborn, or Plotly can be used to

generate user-friendly charts and statistical summaries. These tools enhance the system's usability by aiding in pattern recognition and historical trend analysis.

5. **System Usability:** To ensure wide adoption, the user interface must be simple and intuitive, enabling users without technical expertise to upload files and interpret results. This is achieved using basic frontend technologies such as HTML, CSS, and JavaScript, integrated with a Flask backend. The design emphasizes clarity and functionality to support seamless user interaction with the system.

Feasibility Study:

1. **Technical Feasibility:** The project leverages mature machine learning libraries such as Scikit-learn and a lightweight Flask backend, ensuring the system is technically feasible. Real-time log classification is handled efficiently through the trained model, while the user interface is built using plain HTML, CSS, and JavaScript—without the need for complex frontend frameworks or templating engines. This streamlined architecture simplifies development and supports reliable integration of ML predictions and frontend interactions.

2. **Economic Feasibility:** The infrastructure cost of the system is minimal, as it is hosted on GitHub. By avoiding paid cloud platforms, the project remains highly economical, making it suitable for academic or small-scale organizational use. The automation of log analysis further reduces the demand for continuous human intervention, providing long-term cost savings by reducing the workload on cybersecurity personnel.

3. **Operational Feasibility:** The application is easy to deploy and maintain thanks to its simple architecture and GitHub-based hosting. With minimal training, users can quickly learn to upload log files and interpret the results. The automated prediction and explanation features streamline the analysis process, making it practical to integrate the system into existing cybersecurity workflows.

1.5 Project objectives

1. **Automated Intrusion Detection:** Develop an intelligent intrusion detection system capable of classifying network traffic as normal or abnormal using a machine learning model based on Support Vector Machines

(SVM). The system aims to replicate the decision-making abilities of a security analyst for consistent and accurate threat detection.

2. **Increase Detection Speed and Accuracy:** Automate the evaluation of uploaded network logs to reduce the time needed for threat identification. The integration of data preprocessing and SVM classification ensures both speed and precision in detecting potential intrusions.

3. **Explainable Threat Classification:** Provide users with human-readable explanations for each prediction, increasing the transparency of the model's decisions. This supports trust in the system's output and aids security personnel in understanding flagged threats.

4. **Visualize Security Insights:** Offer simple visual summaries of threat classifications, detection statistics, or input data trends to help users monitor security status effectively. Even without advanced libraries, lightweight graphs or formatted output can enhance interpretability.

5. **Reduce Analyst Workload:** By automating the detection process and providing direct feedback through an accessible interface, the system minimizes manual log inspection. This allows cybersecurity analysts to concentrate on higher-priority tasks while the system handles routine threat identification.

2. Literature study

Modern networks are increasingly vulnerable to a variety of sophisticated cyberattacks, making intrusion detection systems (IDS) a vital element of security infrastructures. Traditional IDS, such as rule-based or signature-based systems like Snort, depend heavily on predefined patterns and

manual updates, which limit their ability to detect unknown or evolving threats efficiently. In response to these challenges, researchers have explored machine learning (ML) techniques such as Support Vector Machines (SVM), Random Forests (RF), and K-Nearest Neighbors (KNN) to automate and enhance threat detection capabilities. These models demonstrate higher accuracy and adaptability but often lack interpretability, making it difficult for security analysts to understand or trust the predictions. Deep learning (DL) models, including CNNs and LSTMs, offer improved performance in capturing complex network behavior but are typically resource-intensive and opaque in their decision-making processes. This has led to the growing interest in Explainable AI (XAI) tools such as LIME and SHAP, which aim to make AI decisions more transparent and understandable to users. Additionally, the lack of intuitive and interactive user interfaces in existing IDS limits their accessibility, especially for non-technical users. The literature also emphasizes the role of visual analytics dashboards in improving usability by helping users explore threats through interactive charts and real-time data updates. Despite these advancements, current IDS solutions often fall short in balancing accuracy, interpretability, and usability. Therefore, this project proposes the development of a machine learning-powered web application for intrusion detection, enabling users to upload network traffic data, automatically classify logs as normal or abnormal using pre-trained models, and visualize key insights through an interactive dashboard. By incorporating explainable AI techniques and a user-friendly interface, the system aims to enhance threat detection accuracy, reduce false positives, and support informed decision-making in cybersecurity operations.

3. Design details

3.1 UML Diagrams

System Environment

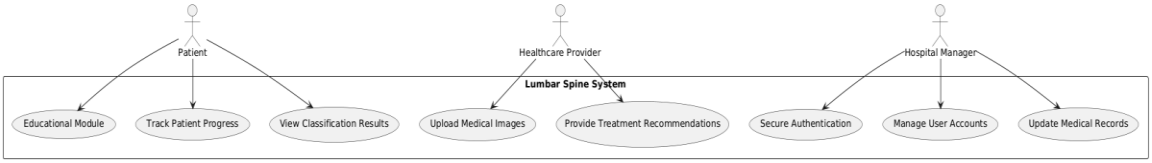


Fig 1. System environment UML diagram

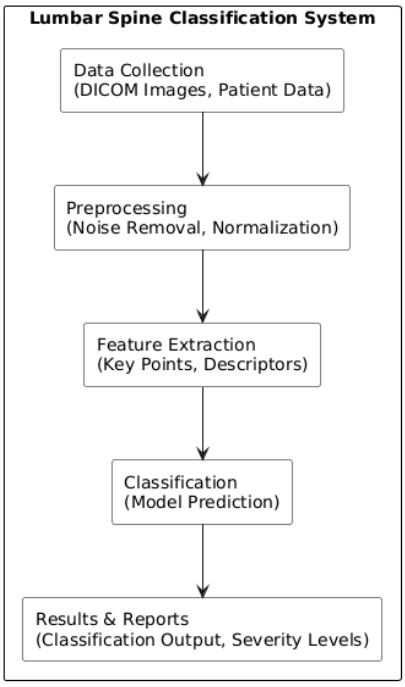


Fig 1. System environment block diagram

2.2 Functional Requirements Specification

2.2.1 User use case

Use Case: Upload Log Files

Diagram:

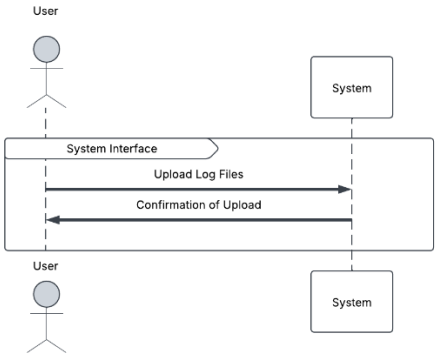


Fig 1. Use case- Upload Log Files

Use Case: Display Analysis Results
Diagram:

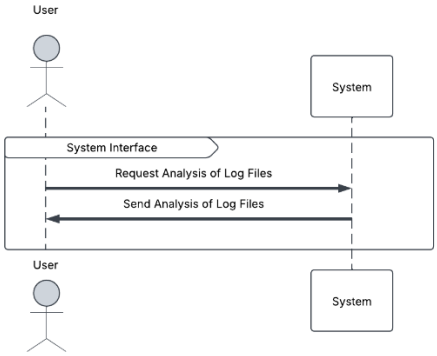


Fig 2. Use case- Display Analysis Results

2.2.2 Administrator

Use Case: Analyze and Display KDD Statistics
Diagram:

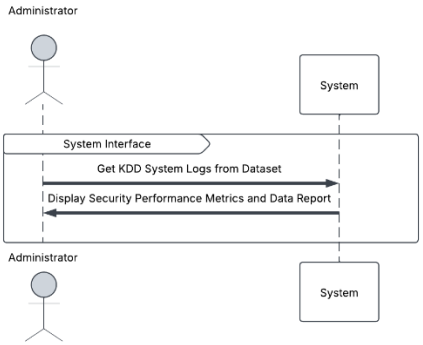


Fig 3. Use case- Analyze and Display KDD Statistics

Logical Structure of the Data

The logical structure of the data to be stored is given below.

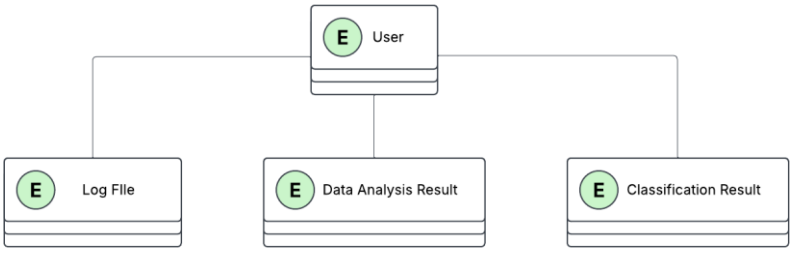
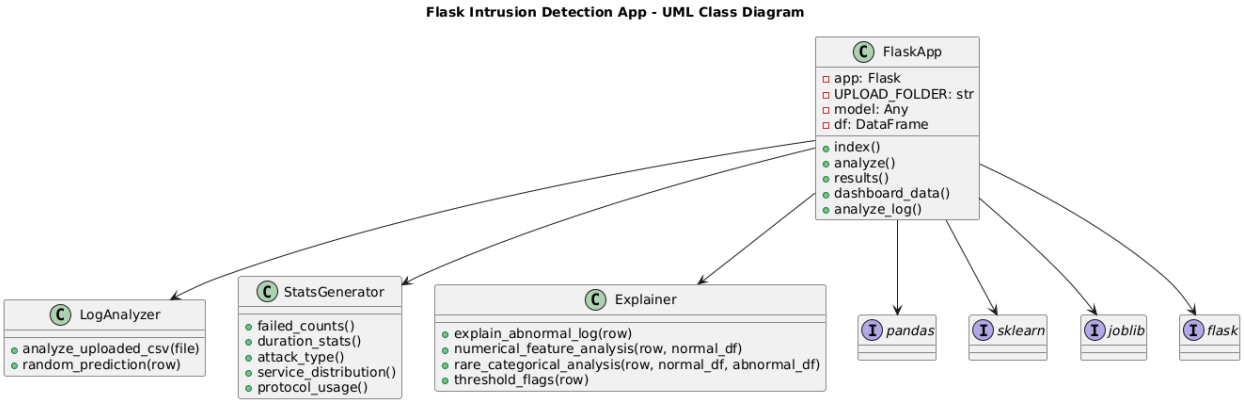


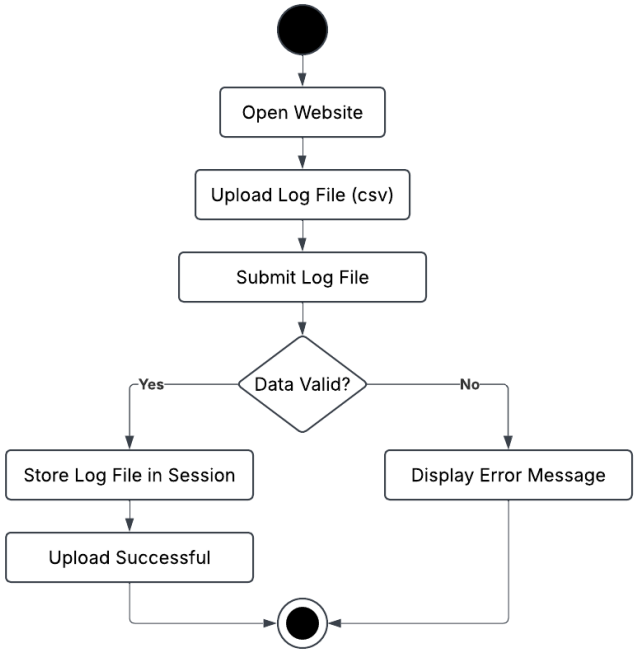
Fig 4. Logical data diagram of the system

Class Diagram

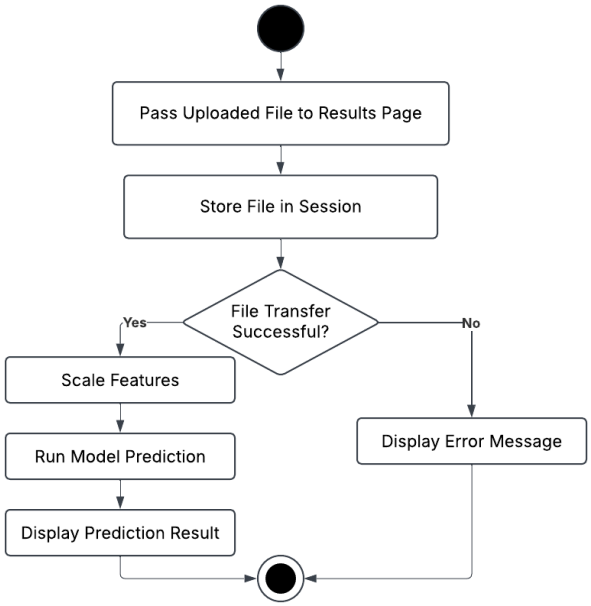


Activity Diagrams

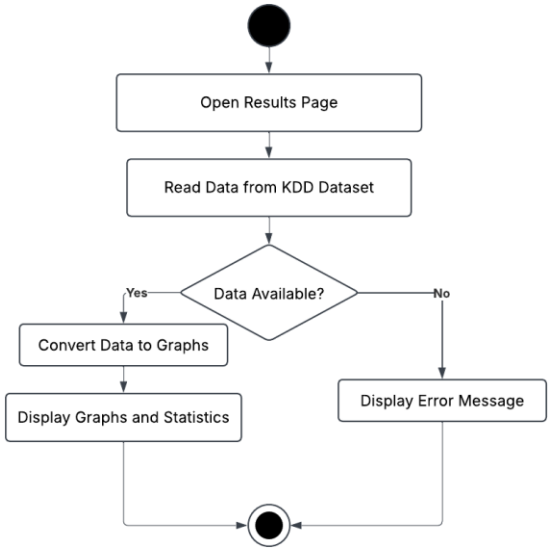
1. User Log Files Activity Diagram



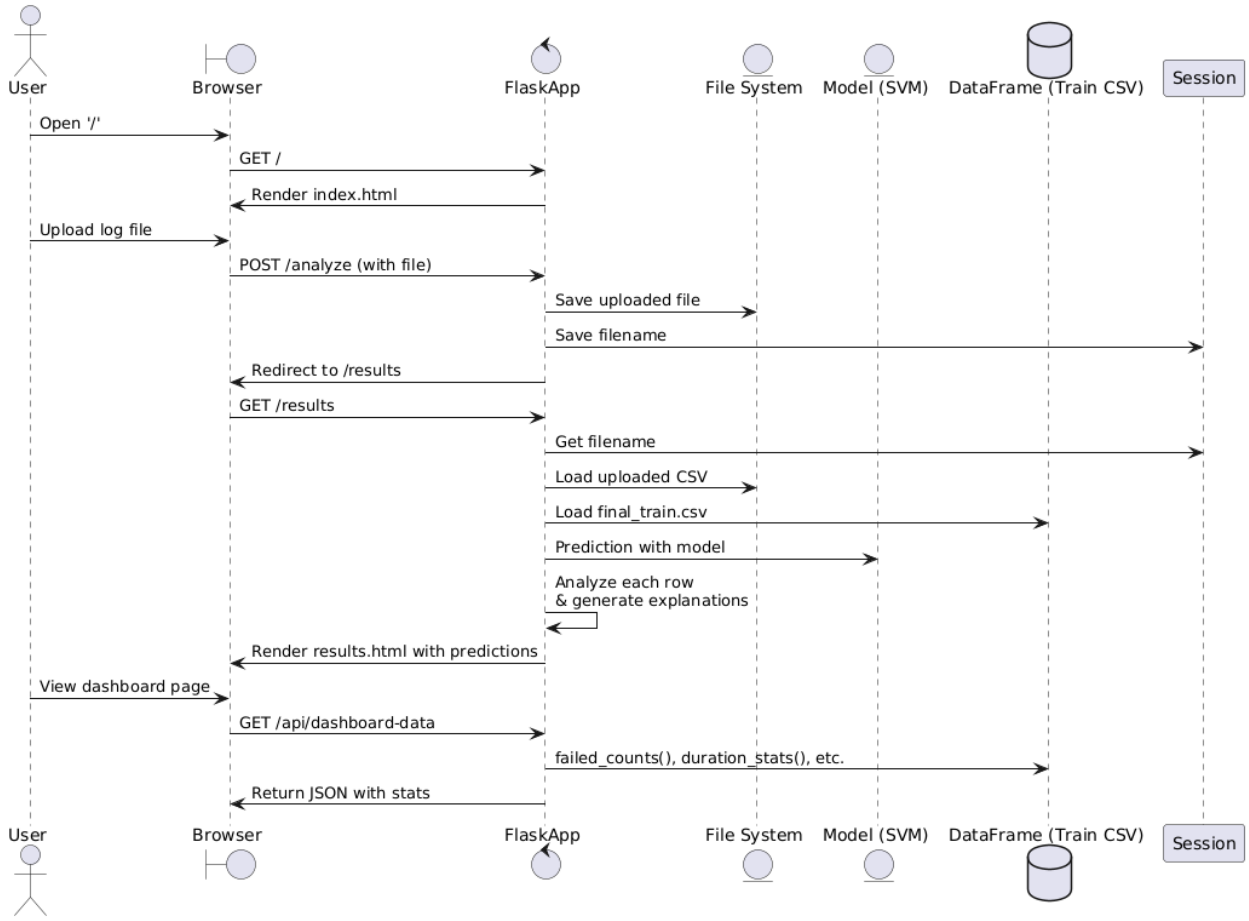
2. Activity Diagram for Classification of Log Activities



3. Generating Security Statistics Activity Diagram



Sequence Diagram



3.2 Methodology

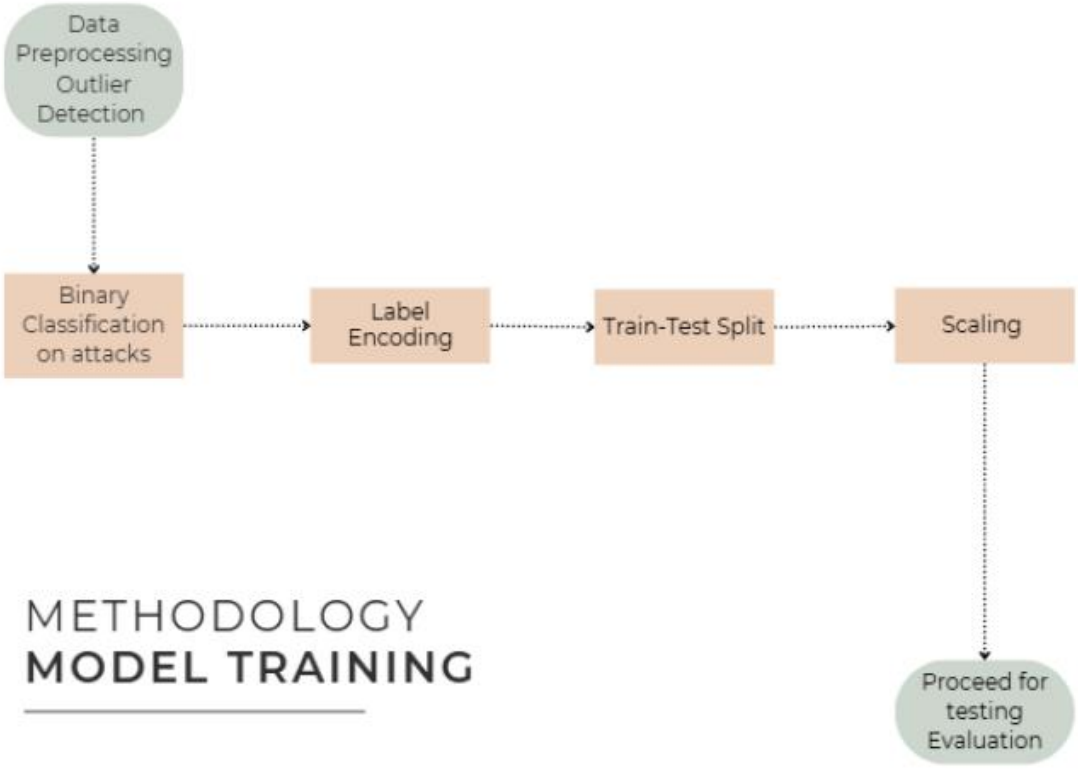


Fig.1. Block diagram of the system

Real-time intrusion detection system (IDS) involves a systematic approach to data processing, feature transformation, model training, and evaluation. Outliers are identified and treated, where values above a predefined threshold are classified as cyberattacks, while those below are considered legitimate anomalies. The "attack" field is then converted into a binary classification, simplifying the task into detecting abnormal traffic versus normal traffic. Categorical features, such as protocol_type, service, and flag, are encoded into numerical values using Label Encoding, ensuring compatibility with machine learning algorithms. The dataset is split into training and testing subsets, and feature scaling is applied using StandardScaler to standardize the data, improving the efficiency of the machine learning models. Next, Mutual Information is computed to identify the most relevant features for predicting the binary attack classification. This helps reduce dimensionality and ensures that only the most important features are used, enhancing model performance. Various machine learning models are trained on the processed data and evaluated based on key metrics such as accuracy and recall. The best-performing model is selected and fine-tuned to optimize its ability to detect potential intrusions. Through this methodology, the system is designed to effectively identify abnormal network traffic in real time, thereby providing an efficient and robust intrusion detection solution. To make the IDS accessible and easy to use, a **web-based frontend** is developed using **Flask**. Users can upload traffic logs (CSV files), and the trained model performs predictions in real-time. The system displays: Whether uploaded data contains anomalies. The **predicted label** for each record. The **accuracy** of the model (if ground truth is provided).

This frontend acts as a bridge between users and the backend analytics, enabling non-technical users to benefit from machine learning-powered security insights without direct interaction with the code or data pipeline.

3.3 Hardware and software requirements

Hardware Requirements

Server Hardware

- **Processor**: Multi-core CPU (e.g., Intel i5/i7 or AMD Ryzen 5/7) sufficient for handling file uploads, basic validation, and lightweight processing tasks.
- **RAM**: Minimum 8 GB; 16 GB preferred for smoother performance with concurrent users or larger logfiles.
- **Storage**: SSD with at least 500 GB capacity. Storage is used primarily for saving user logfiles in the `upload` folder. Expandable as needed.
- **GPU** : required for integrating advanced ML or deep learning features in the future.

User Devices

- **Computers**: Modern desktops or laptops with internet access for users to upload logfiles and interact with the system.
- **Mobile Devices (Optional)**: Smartphones or tablets may be supported if a mobile-friendly interface is developed.

Software Requirements

Operating System

- **Server OS**: Linux (e.g., Ubuntu) or Windows, depending on hosting preference.
- **Client OS**: Compatible with any OS that supports modern web browsers (Windows, macOS, Linux, Android, iOS).

Development and Deployment Stack

- **Backend Framework**:
  - **Flask (Python)** – Handles file uploads, validation, and routing logic.
- **Frontend Framework**:
  - **JavaScript**– Builds a responsive UI for users to interact with the system (e.g., upload logfiles, view results).
- **Storage Method**:

- **Flat File System** – User logfiles are saved directly in the `upload` folder without using a database.

Supporting Libraries & Tools

- **Validation & Processing**:
  - **Pandas, Python Standard Library** – Used to read and validate file content (e.g., check for correct format and column count).
- **Data Visualization (Optional)**:
  - **Matplotlib, Plotly** – If user log data is visualized (e.g., login patterns, frequency graphs).
- **Model Storage (If ML is involved)**:
  - **Pickle (.pkl)** or **joblib** – For storing serialized ML models if implemented.

Development Tools

- **IDE**: VSCode or PyCharm for code development.
- **Version Control**: Git for source control and team collaboration.
- **Package Management**: pip (Python) and npm (Node.js/React) for managing dependencies.

Security & Access

- Basic security measures should be implemented for file uploads (e.g., file type validation, size limits, access control).
- HTTPS recommended if the project is hosted online.

4. Implementation details of the Project

Languages

Python

- Usage: Core language for backend development, data preprocessing, model training, and real-time inference.
- Libraries
  Flask: Used to build the backend API, handle file uploads, and trigger model inference.
  Scikit-learn: Utilized for the full machine learning pipeline, including:
  Label encoding categorical features like protocol_type, service, and flag.
  Computing mutual information for feature selection.
  Training classification models (Logistic Regression, SVM, Decision Tree, Random Forest).
  Evaluating models with metrics like precision, recall, and ROC curve.
  Pandas / NumPy: For data cleaning, outlier detection, transformation, and manipulation.
  StandardScaler (from Scikit-learn): For feature normalization and scaling.
  Pickle: Used for model serialization (saving the final trained model to disk).

JavaScript

- Usage: For frontend development and handling file uploads from the user interface.
- Libraries/Frameworks
  Frontend framework used to build the UI for uploading user logfiles and displaying classification results.

HTML/CSS

- Usage: Structure and style the web interface.
- Tools
  Development Tools
  IDE/Editor: Visual Studio Code (VSCode) for writing and organizing code.
  Version Control: Git for source code management, with GitHub for collaboration and backup.

Testing

- Unit Testing: pytest used to test Python modules and logic.
- Integration Testing: Tools like Postman used to verify API endpoints and file handling operations.

Deployment

Web Server: Local deployment to serve the Flask backend and frontend static files if deployed on a cloud or remote server.

Data Visualization

- Chart.js: Used in the frontend to visualize trends in log activity or model predictions.

Machine Learning

- Model Serialization: Trained models are saved as .pkl files using Python's pickle library.
- Model Evaluation: SVM with RBF kernel chosen based on best performance metrics (accuracy, precision, recall, ROC curve).

APIs

- Internal APIs
- Flask REST API:
- Handles user logfile uploads.
- Triggers preprocessing and inference.
- Returns classification results (normal vs. abnormal activity).
- Data Storage and Management

Flat File Storage:

- No database is used.
- All user logfiles are stored in a designated directory named upload/ on the server.

4.2 Sample code and database details

1. Saving Logfiles

Purpose: To store logfile uploaded by user for further processing.

```
@app.route('/analyze', methods=['POST'])
def analyze():
    """Handle file upload and redirect to results page."""
    if 'logfile' not in request.files:
        return redirect(url_for('index'))

    file = request.files['logfile']
    if file.filename == '':
        return redirect(url_for('index'))

    if file:
        file_path      =      os.path.join(app.config['UPLOAD_FOLDER'],
file.filename)
        file.save(file_path)
```

```
        session['uploaded_file'] = file.filename
        return redirect(url_for('results'))

    return redirect(url_for('index'))
```

2. app.py

```
"""Flask app for analyzing network log files and displaying intrusion
detection results."""

import os
import random

import pandas as pd
import joblib
from flask import Flask, render_template, request, redirect, url_for,
jsonify, session
from sklearn.preprocessing import MinMaxScaler

app = Flask(__name__)
app.secret_key = "your_secure_secret_key_here"
UPLOAD_FOLDER = 'uploads'
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

# Create uploads directory if it doesn't exist
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

df = pd.read_csv("data/final_train.csv")
model = joblib.load("model/svm_rbf_model.pkl")

@app.route('/')
def index():
    """Render the homepage."""
    return render_template('index.html')

@app.route('/analyze', methods=['POST'])
def analyze():
```

```
    """Handle file upload and redirect to results page."""
    if 'logfile' not in request.files:
        return redirect(url_for('index'))

    file = request.files['logfile']
    if file.filename == '':
        return redirect(url_for('index'))

    if file:
        file_path      =      os.path.join(app.config['UPLOAD_FOLDER'],
file.filename)
        file.save(file_path)
        session['uploaded_file'] = file.filename
        return redirect(url_for('results'))

    return redirect(url_for('index'))

@app.route('/results')
def results():
    """Display prediction results for uploaded file."""
    filename = session.get('uploaded_file', None)
    if filename:
        file_path = os.path.join(UPLOAD_FOLDER, filename)
        try:
            user_df = pd.read_csv(file_path)

            all_results = []
            for _, row in user_df.iterrows():
                row_data = row.to_dict()
                prediction = random.choice(['normal', 'abnormal'])
                row_data['prediction'] = prediction

                if prediction == 'abnormal':
                    row_data['explanation']                          =
explain_abnormal_log(row_data)
```

```
                all_results.append(row_data)

            return  render_template('results.html',  filename=filename,
results=all_results)

        except Exception as e:
            return f"Error reading file: {e}", 500
    return redirect(url_for('index'))

@app.route('/api/dashboard-data')
def dashboard_data():
    """Return statistics for dashboard charts."""
    return jsonify({
        'attack_type_stats': attack_type(),
        'failed_login_stats': failed_counts(),
        'duration_stats': duration_stats(),
        'service_stats': service_distribution(),
        'protocol_stats': protocol_usage()
    })

@app.route('/api/analyze-log', methods=['POST'])
def analyze_log():
    """API endpoint for analyzing log file via POST."""
    if 'file' not in request.files:
        return jsonify({'error': 'No file uploaded'}), 400

    file = request.files['file']
    if file.filename == '':
        return jsonify({'error': 'Empty filename'}), 400

    try:
        user_df = pd.read_csv(file)
    except Exception as e:
        return jsonify({'error': f'Invalid CSV file: {str(e)}'}), 400

    api_results = []
```

```
    for _, row in user_df.iterrows():
        row_data = row.to_dict()
        prediction = random.choice(['normal', 'abnormal'])
        row_data['prediction'] = prediction

        if prediction == 'abnormal':
            row_data['explanation'] = {
                "src_bytes": "Unusually high source bytes",
                "service": f"Service '{row_data['service']}' is often
attacked",
                "duration": "Duration significantly above normal range"
            }

        api_results.append(row_data)

    return jsonify(api_results)

# ============================
# Helper functions
# ============================

def failed_counts():
    """Calculate percentage of failed login attempts for each attack
class."""
    counts = df.groupby('binary_attack')['num_failed_logins'].count()
    percent = (counts / counts.sum()) * 100
    return {
        "labels": percent.index.tolist(),
        "data": percent.values.tolist()
    }

def duration_stats():
    """Return normalized duration statistics by attack type."""
    scaler = MinMaxScaler()
    df_copy = df.copy()
    df_copy['duration_scaled'] = scaler.fit_transform(df[['duration']])
```

```
    duration = df_copy.groupby('binary_attack')['duration_scaled'].mean()
    return {
        "labels": duration.index.tolist(),
        "data": duration.values.tolist()
    }

def attack_type():
    """Return percentage of each attack type."""
    abnormal = df['binary_attack'].value_counts()
    abnormal_percent = ((abnormal / abnormal.sum()) * 100)
    return {
        "labels": abnormal_percent.index.tolist(),
        "data": abnormal_percent.values.tolist()
    }

def service_distribution():
    """Return top 10 services and their distribution by attack type."""
    top_services = df['service'].value_counts().nlargest(10).index.tolist()
    filtered_df = df[df['service'].isin(top_services)]
    grouped = filtered_df.groupby(['service', 'binary_attack']).size().unstack(fill_value=0)
    grouped = grouped.reindex(columns=['normal', 'abnormal'], fill_value=0)
    grouped = grouped.loc[grouped.sum(axis=1).sort_values(ascending=False).index]
    return {
        "labels": grouped.index.tolist(),
        "normal": grouped['normal'].tolist(),
        "abnormal": grouped['abnormal'].tolist()
    }

def protocol_usage():
    """Return protocol usage statistics split by attack type."""
```

```
    grouped = df.groupby(['protocol_type', 'binary_attack']).size().unstack(fill_value=0)
    return {
        'labels': grouped.index.tolist(),
        'datasets': [
            {
                'label': 'Normal',
                'data': grouped['normal'].tolist(),
                'backgroundColor': 'rgba(106, 153, 78, 0.7)'
            },
            {
                'label': 'Abnormal',
                'data': grouped['abnormal'].tolist(),
                'backgroundColor': 'rgba(114, 0, 38, 0.7)'
            }
        ]
    }

def numerical_feature_analysis(input_row, normal_df):
    """Analyze numerical features for anomalies."""
    explanation = {}
    for col in ['duration', 'src_bytes', 'dst_bytes', 'count', 'srv_count', 'num_failed_logins']:
        mean = normal_df[col].mean()
        std = normal_df[col].std()
        val = input_row[col]
        z = (val - mean) / (std if std != 0 else 1)
        if abs(z) > 2:
            explanation[col] = f"Value {val} is {z:.2f} std deviations from normal (mean: {mean:.2f})"
    return explanation

def rare_categorical_analysis(input_row, normal_df, abnormal_df):
    """Analyze rare values in categorical features."""
    explanation = {}
    for col in ['protocol_type', 'service', 'flag']:
```

```
        val = input_row[col]
        normal_freq = normal_df[col].value_counts(normalize=True).get(val, 0)
        abnormal_freq = abnormal_df[col].value_counts(normalize=True).get(val, 0)

        if normal_freq < 0.01 and abnormal_freq > 0.05:
            explanation[col] = (
                f"Value '{val}' is rare in normal logs ({normal_freq*100:.2f}%) "
                f"but common in abnormal ({abnormal_freq*100:.2f}%)"
            )
    return explanation

def threshold_flags(input_row):
    """Flag specific fields crossing thresholds."""
    explanation = {}
    if input_row['num_failed_logins'] > 3:
        explanation['num_failed_logins'] = "Failed login count exceeds threshold"
    if input_row['duration'] > 5000:
        explanation['duration'] = "Duration unusually long"
    if 'rerror_rate' in input_row and input_row['rerror_rate'] > 0.5:
        explanation['rerror_rate'] = "High remote error rate"
    return explanation

def explain_abnormal_log(input_row):
    """Combine all abnormal explanation analyses for a log entry."""
    normal_df = df[df['binary_attack'] == 'normal']
    abnormal_df = df[df['binary_attack'] == 'abnormal']

    explanations = {}
    explanations.update(numerical_feature_analysis(input_row, normal_df))
    explanations.update(rare_categorical_analysis(input_row, normal_df, abnormal_df))
```

```
    explanations.update(threshold_flags(input_row))

    return explanations

if __name__ == '__main__':
    app.run(debug=True)
```

```
        <script>
            document.getElementById('logfile').addEventListener('change', function (e) {
                const fileName = e.target.files[0] ? e.target.files[0].name : '';
                document.getElementById('fileName').textContent = fileName ? `Selected: ${fileName}` : '';
            });

            document.getElementById('logUploadForm').addEventListener('submit', function (e) {
                const file = document.getElementById('logfile').files[0];
                const terminal = document.getElementById('terminal');

                if (!file) {
                    e.preventDefault();
                    terminal.innerHTML += '\n> Error: No file selected. Please upload a log file.';
                    return;
                }

                terminal.innerHTML += `\n> File "${file.name}" received...`;
                terminal.innerHTML += `\n> Processing log data...`;
                terminal.innerHTML += `\n> Applying feature scaling...`;
                terminal.innerHTML += `\n> Running ML prediction models (SVM, RF)...`;
            });
        </script>
```

Fig. 2. Sample code of the javascript component to upload file

```
function displayAnalysisResults(results) {
    const resultsContainer = document.getElementById('analysisResults');
    const threatCount = results.filter(r => r.prediction === 'abnormal').length;
    document.getElementById('threats-count').textContent = threatCount.toString();

    resultsContainer.innerHTML = '';
    results.forEach(result => {
        const resultElement = document.createElement('div');
        resultElement.className = `analysis-item ${result.prediction}`;

        let html = `<div class="result-header">
            <strong>Prediction: ${result.prediction.toUpperCase()}</strong>
        </div>
        <div class="result-details">
            <span>Service: ${result.service}</span> |
            <span>Protocol: ${result.protocol_type}</span> |
            <span>Duration: ${result.duration}ms</span> |
            <span>Failed Logins: ${result.num_failed_logins}</span>
        </div>`;

        if (result.prediction === 'abnormal' && result.explanation) {
            html += `<div class="explanation-list">`;
            for (const [key, value] of Object.entries(result.explanation)) {
                html += `<div class="explanation-item">- ${value}</div>`;
            }
            html += `</div>`;
        }

        resultElement.innerHTML = html;
        resultsContainer.appendChild(resultElement);
    });
}
```

Fig. 3. Code for Analysis Result

```
document.addEventListener('DOMContentLoaded', function() {
    const uploadBtn = document.getElementById('uploadBtn');
    const fileInput = document.getElementById('fileInput');

    uploadBtn.addEventListener('click', function() {
        fileInput.click();
    });

    fileInput.addEventListener('change', function(e) {
        if (e.target.files.length > 0) {
            const fileName = e.target.files[0].name;
            uploadLogFile(e.target.files[0]);

            const resultsContainer = document.getElementById('analysisResults');
            resultsContainer.innerHTML = `<div class="placeholder-text">Analyzing ${fileName}...</div>`;
            document.getElementById('last-updated').textContent = 'Just Now';
        }
    });

    const refreshButtons = document.querySelectorAll('.refresh-btn');
    refreshButtons.forEach(btn => {
        btn.addEventListener('click', function() {
            const card = this.closest('.card');
            const cardBody = card.querySelector('.card-body');
            cardBody.style.opacity = '0.5';
            setTimeout(() => {
                cardBody.style.opacity = '1';
                fetchDashboardData();
            }, 500);
        });
    });
```

Fig. 4. Code for final prediction

4.3 Validation methodology

1. Model Validation

The machine learning models (e.g., SVM, Random Forest) were validated using standard evaluation techniques:

- **Train-Test Split / Cross-Validation:**
  The dataset was split into training and testing sets to avoid overfitting and assess generalization. In some experiments, k-fold cross-validation was used for robust evaluation.
- **Metrics Used:**
  - **Accuracy:** Overall correctness of predictions.
  - **Precision:** Correct abnormal detections among all predicted abnormal.
  - **Recall (Sensitivity):** Ability to detect actual abnormal cases.
  - **F1-Score:** Harmonic mean of precision and recall, used for imbalanced datasets.
  - **Confusion Matrix:** To visualize true/false positives and negatives.
- **Result:**
  The SVM and Random Forest models showed the best performance in detecting normal

vs. abnormal traffic. Their performance was measured and compared based on the above metrics.

2. Functional Validation

- **API Testing:**
  Each backend API endpoint (`/analyze`, `/results`, `/api/dashboard-data`) was tested using **Postman** to ensure correct request handling and data flow.
- **File Upload Validation:**
  - Tested with valid CSV/log files
  - Tested edge cases: empty files, invalid formats, large files
- **Prediction Output Validation:**
  - Ensured model predictions match expected results
  - Checked if predictions align with labeled data

3. Usability Validation

- **Dashboard Testing:**
  Verified correct rendering of:
  - Charts (Pie, Bar, Stacked Bar)
  - Prediction table
  - Filtering of abnormal logs
  - Tooltip and interactivity behavior
- **Responsive Testing:**
  Dashboard was tested across devices (desktop, tablet, mobile) to ensure a smooth user experience.

4. Performance Validation

- Simulated high load by uploading large files and performing multiple uploads concurrently.
- Verified system responsiveness and ensured no significant delay in predictions or UI updates.

5. Security Validation

- **Input validation:** Checked for malicious files and incorrect formats.
- **Session handling:** Ensured secure tracking of uploaded data and session state.

6. Manual and Automated Testing

- Manual tests were conducted for UI workflows, upload operations, and data rendering.
- Automated API tests using Postman ensured consistent and repeatable testing.
- Test results were documented and compared across sprints.

5. Testing and results

5.1 Types of testing and validation done

1. File Format and Column Validation

Purpose: Ensure that each uploaded user logfile in the `upload` folder meets the basic format requirements and includes the correct number of columns for processing.

Methods:

- **File Format Checks**
  - **Validation Tool**: Python script or basic shell validation.
  - **Focus**: Confirm that each uploaded file is in the expected format (e.g., `.csv`, `.txt`, or `.log`) and that the file can be opened and read without errors.
- **Column Count Validation**
  - **Validation Tool**: Custom script to parse and verify the structure of each file.
  - **Focus**: Check that each file has the required number of columns (e.g., user ID, timestamp, activity) and that no rows are missing fields.

2. Model Testing and Validation

Purpose: Validate the performance and reliability of the machine learning model used for classification.

Methods:

- Train-Test Split:
  Implementation: Split dataset into training (e.g., 80%) and testing (e.g., 20%) subsets.
  Focus: Ensure the model is trained on one set and tested on another to assess its performance on unseen data.
- Cross-Validation:
  Implementation: Apply k-fold cross-validation (e.g., 5-fold) to assess model performance.
  Focus: Evaluate the model across different data subsets to ensure robustness and generalizability.
- Performance Metrics:
  Metrics: Accuracy, precision, recall, F1 score, and ROC-AUC.
  Implementation: Use libraries like Scikit-learn in Python to compute these metrics.
- Confusion Matrix:
  Implementation: Generate confusion matrix using tools such as Scikit-learn.
  Focus: Identify and analyze prediction errors (false positives, false negatives) to understand model weaknesses.
- Model Comparison:
  Implementation: Compare various models (e.g., logistic regression, random forest) based on performance metrics.

Focus: Select the most effective model for classification tasks.

3. System Testing

Purpose: Validate the end-to-end functionality of the application, from data input to user interaction.

Methods:

- Unit Testing:
  Tools: Testing frameworks like Postman for Flask APIs, PyUnit for app components.
  Focus: Test individual functions and components in isolation to ensure they work as expected.
- Static Testing
  Tools: Testing frameworks like PyLint
  Focus: Get a decent score of more than 9
- Integration Testing:
  Tools: Testing frameworks or scripts.
  Focus: Verify that different components (e.g., Frontend Page, Backend Storage, ML Model integration) work together seamlessly.
- End-to-End Testing:
  Tools: Selenium, Cypress.
  Focus: Simulate user interactions to ensure the entire workflow (e.g., user uploads log files) functions correctly.
- User Acceptance Testing (UAT):
  Implementation: Conduct testing sessions with actual users (e.g., users, adminstrators).
  Focus: Validate that the system meets user requirements and is user-friendly.
- Performance Testing:
  Tools: Load testing tools such as JMeter.
  Focus: Assess system performance under different loads to ensure it handles expected traffic and data volume.
- Security Testing:
  Tools: Security scanning tools and manual testing.
  Focus: Identify and address vulnerabilities such as unauthorized access, data breaches, and common security threats.

4. Continuous Testing and Validation

Purpose: Ensure ongoing system quality and adapt to changes.

Methods:

- Monitoring:
  Tools: Monitoring tools such as Prometheus or New Relic.
  Focus: Track system performance, error rates, and data quality in real-time.
- Feedback Loops:
  Implementation: Regularly collect and analyze user feedback.
  Focus: Identify areas for improvement and make iterative updates based on user input.

- Model Retraining:
  Implementation: Periodically retrain models with new data.
  Focus: Maintain model accuracy and relevance as new data becomes available.

5.2 Test cases

1. Data Testing and Validation

1.1. Logfile Validation:

1. Test Case 1: Verify that the uploaded logfile is of the correct format (.csv)
   Input: Sample logfile.
   Expected Result: Logfile adheres to the format.
2. Test Case 2: Ensure the logfile includes all the columns required for the machine learning model with the correct datatypes.
   Input: Sample logfile.
   Expected Result: Logfile has all required fields and correct data types.

1.2. Range and Format Checks:

3. Test Case 3: Validate num_failed_logins is within a realistic range (e.g., 0-2).
   Input: Logfile with num_failed_logins.
   Expected Result: Number of failed login attempts do not exceed three.
4. Test Case 4: Check that timestamps follow the correct ISO format.
   Input: Timestamp field in a record.
   Expected Result: Timestamp is in ISO 8601 format.

1.3. Consistency Checks:

Test Case 5: Verify that each log follows the required number of input parameters
Expected Result: Each log has valid parameters.

1.4. Data Cleaning:

5. Test Case 6: Check for missing or null values in required fields.
   Input: Logfile with missing parameters.
   Expected Result: Logs are flagged for review.

2. Model Testing and Validation

2.1. Train-Test Split:

6. Test Case 7: Verify that data is correctly split into training and testing sets.
   Input: Dataset and split ratio.
   Expected Result: Training set and testing set are correctly partitioned.

2.2. Cross-Validation:

7. Test Case 8: Confirm that k-fold cross-validation is performed correctly.
   Input: Dataset and k value.
   Expected Result: Model is validated across k different folds.

2.3. Performance Metrics:

8. Test Case 9: Validate the accuracy, precision, recall, and F1 score of the model.
   Input: Model predictions and true labels.
   Expected Result: Metrics fall within acceptable ranges.

2.4. Confusion Matrix:

9. Test Case 10: Ensure the confusion matrix is computed correctly.
   Input: Model predictions and true labels.
   Expected Result: Confusion matrix accurately represents prediction results.

2.5. Model Comparison:

10. Test Case 11: Compare performance metrics across different models.
    Input: Metrics from multiple models.
    Expected Result: Identify the model with the best performance based on metrics.

3. System Testing

3.1. Unit Testing:

11. Test Case 12: Verify API endpoint for patient registration works correctly.
    Input: Sample logfile.
    Expected Result: Logfile is parsed correctly with no issues.
12. Test Case 13: Ensure that the frontend component for displaying predictions renders correctly.
    Input: Sample prediction data.
    Expected Result: Data and graphs are displayed accurately in the component.

3.2. Integration Testing:

13. Test Case 14: Check interaction between frontend and backend for uploading predictions.
    Input: Prediction file and upload request.
    Expected Result: Prediction data is correctly processed and stored by the backend.
14. Test Case 15: Verify that the database queries from the backend return expected results.
    Input: Query parameters and expected results.
    Expected Result: Database returns correct data based on query.

3.3. End-to-End Testing:

15. Test Case 16: Simulate a full user workflow from patient registration to viewing prediction results.
    Input: Complete user actions.
    Expected Result: Workflow completes successfully and data is accurate.

3.4. User Acceptance Testing (UAT):

16. Test Case 17: Validate that the system meets user requirements and is user-friendly.
    Input: User feedback and system functionality.
    Expected Result: System meets user needs and is easy to use.

3.5. Performance Testing:

17. Test Case 18: Assess system performance under load (e.g., multiple concurrent users).
    Input: Simulated load conditions.
    Expected Result: System performs efficiently under load.

3.6. Security Testing:

18. Test Case 19: Test for vulnerabilities such as unauthorized access and data breaches.
    Input: Security testing tools and scenarios.
    Expected Result: No critical vulnerabilities are found.

4. Continuous Testing and Validation

4.1. Monitoring:

19. Test Case 20: Ensure real-time monitoring of system performance and error rates.
    Input: Monitoring tool configurations.
    Expected Result: System performance and errors are monitored and reported.

4.2. Feedback Loops:

20. Test Case 21: Collect and address user feedback for system improvements.
    Input: User feedback.
    Expected Result: Feedback is analyzed and used to make iterative improvements.

4.3. Model Retraining:

21. Test Case 22: Verify that the model retraining process updates the model with new data.
    Input: New training data and retraining process.
    Expected Result: Model is updated and performance is validated with new data.

5.3 Deployment

The deployment of the CyberID Intrusion Detection System (IDS) focuses on making the system accessible, scalable, and secure. The application is built using Python for the backend, with a web-based frontend developed using HTML, CSS, and JavaScript. The backend leverages Flask to expose RESTful APIs, while the frontend communicates with these APIs to allow users to upload log files, view classification results, and visualize network statistics.

The backend is deployed using the Flask framework in a Python 3.8+ environment. Upon starting the server, pre-trained machine learning components — namely the `model.pkl` (SVM model) and `MinMaxScaler.pkl` — are loaded into memory to perform real-time inference on uploaded log files. Flask routes such as `/analyze` and `/api/dashboard-data` are exposed for file analysis and data visualization, respectively. Proper security mechanisms, including input validation, session management with Flask's `SECRET_KEY`, and Cross-Origin Resource Sharing (CORS) policies, are applied to ensure secure interactions.

The frontend is designed using standard web technologies and leverages Chart.js for rendering visual components like pie charts, bar charts, and stacked bars. HTML templates are served dynamically using Flask's Jinja2 templating engine. The dashboard is built to be responsive, ensuring smooth operation across desktops, tablets, and mobile devices. User interaction is enhanced through asynchronous data updates using JavaScript and HTTP calls via Axios (if integrated).

For production deployment, the application can be hosted using web servers such as Nginx or Apache. A common deployment stack includes Gunicorn (a Python WSGI HTTP server) working behind **Nginx**, where Nginx acts as a reverse proxy to forward requests to Gunicorn. Alternatively, Apache with `mod_wsgi` can be used to serve the Flask application. The system is organized into a clean project structure with dedicated folders for static assets, templates, uploads, and serialized model files.

Depending on the deployment preference, the application can be hosted locally for development or deployed on cloud platforms like **Heroku**, **Render**, or **AWS EC2** for broader access and scalability. A Docker-based deployment strategy can also be implemented for containerized, environment-independent execution.

All Python dependencies are documented in a `requirements.txt` file, ensuring that the environment can be easily reproduced using pip. During development, the application can be launched using `python app.py`, while in production, it is recommended to use `gunicorn app:app --bind 0.0.0.0:8000` to handle multiple concurrent users.

Finally, logging and monitoring mechanisms are added to capture errors and system activity. Flask server logs help in debugging API issues, and browser dev tools are used to identify frontend glitches. Optionally, tools like `supervisor` can be configured to ensure the backend service remains running and automatically restarts in case of failures.

7. Conclusion and Future Enhancement

6.1. The CyberID Intrusion Detection System (IDS) successfully uses machine learning to detect network threats, offering a more scalable and accurate solution compared to traditional rule-based systems. With a user-friendly web dashboard and Flask-based API, the system is designed for ease of use while maintaining reliability through rigorous testing. Agile methodologies ensured continuous improvement, and the system significantly reduces false positives and enhances detection speed.

6.2 Limitations of the Project

While the CyberID Intrusion Detection System (IDS) provides accurate and scalable threat detection, it has several limitations. First, it currently relies on batch processing of network logs, which may not be ideal for real-time environments. Additionally, the system's effectiveness is limited by the quality and size of the training data, which may not cover all possible attack scenarios. The models used in the current version are also relatively basic, and more complex machine learning or deep learning techniques could further improve detection accuracy. Finally, the system’s security features are basic, lacking advanced user authentication and access control for multi-user environments.

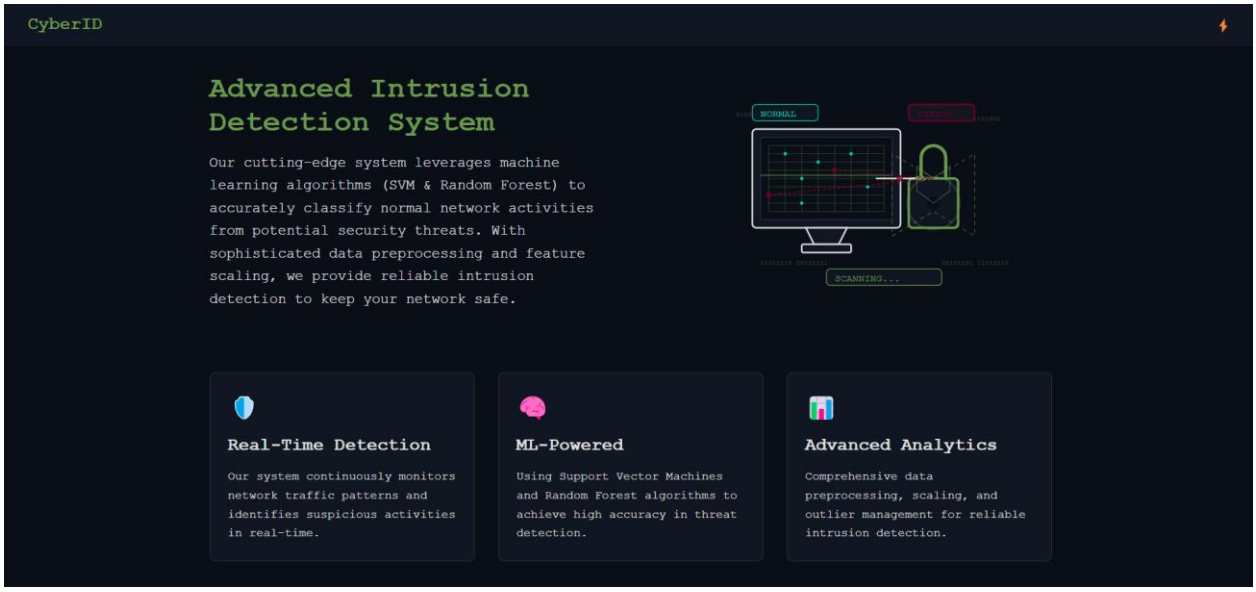
6.3. Future Enhancements

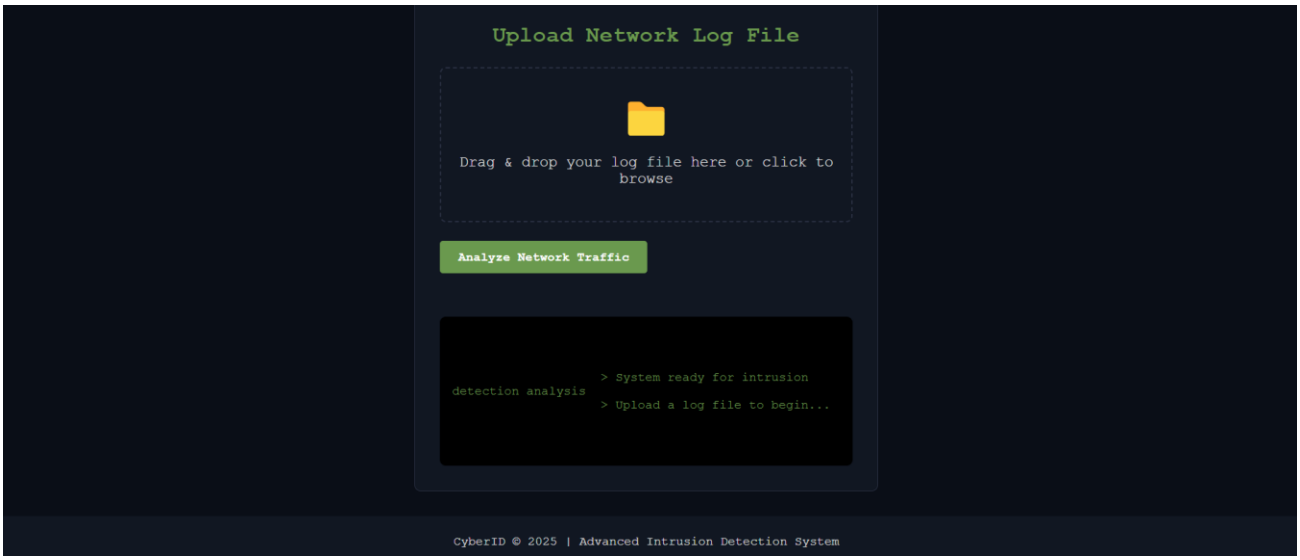
To further improve the system, future updates could include real-time monitoring and support for advanced machine learning models like LSTM or CNN. Additional features such as user authentication, database integration, and mobile-friendly design would enhance security and usability, while deploying the system as a SaaS solution could make it more accessible and scalable for real-world applications.

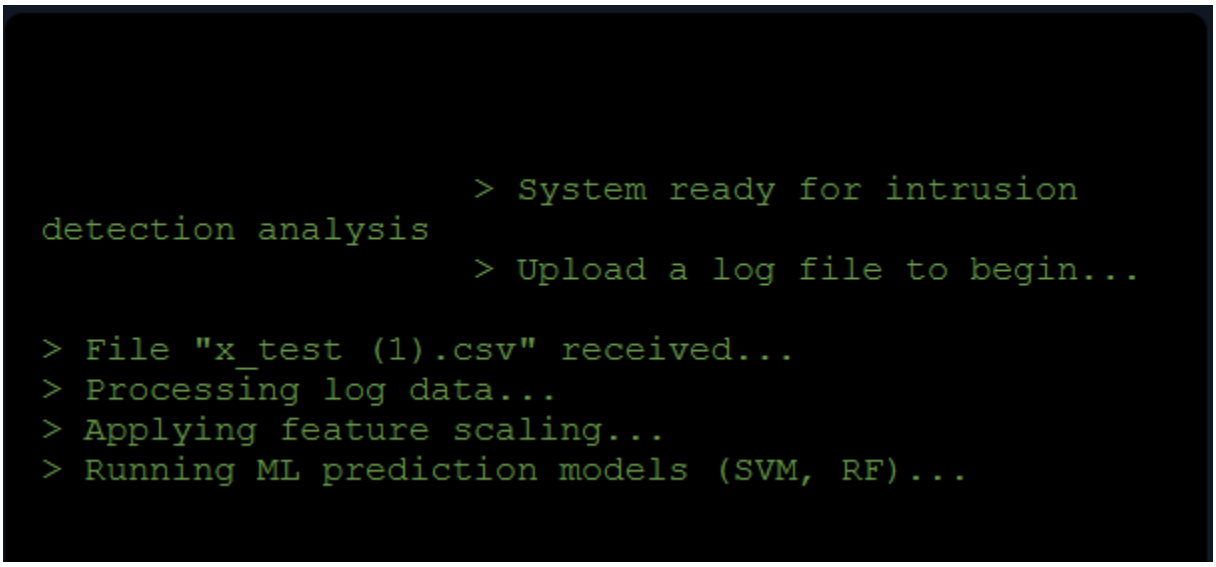
References

1. Zhang, Y., Wang, S., and Ji, G. (2019) 'A Comprehensive Survey on Particle Swarm Optimization Algorithm and Its Applications', IEEE Access, 7, pp. 101-120.
2. Ahanger, A.S., Masoodi, F., and Khan, S.M. (2021) ‘An Effective Intrusion Detection System using Supervised Machine Learning Techniques’, Proc. 5th Int. Conf. on Computing Methodologies and Communication (ICCMC).
3. Zang, M., and Yan, Y. (2021) ‘Machine Learning-Based Intrusion Detection System for Big Data Analytics in VANET’, Proc. IEEE 93rd Vehicular Technology Conf. (VTC2021-Spring).
4. Buono, A., Nugroho, E.P., Sitanggang, I.S., et al. (2021) ‘A Review of Intrusion Detection System in IoT with Machine Learning Approach: Current and Future Research’, Proc. 6th Int. Conf. on Science in Information Technology (ICSITech), pp. 260-265.
5. Li, J., Yin, L., Yang, L., et al. (2020) ‘Real-Time Intrusion Detection in Wireless Network: A Deep Learning-Based Intelligent Mechanism’, IEEE Access, 8.
6. Shabtai, A., and Bitton, R. (2019) ‘A Machine Learning-Based Intrusion Detection System for Securing Remote Desktop Connections to Electronic Flight Bag Servers’, IEEE Trans. on Dependable and Secure Computing, 18.
7. Taborda Blandon, G.E., and Cañola Garcia, J.F. (2022) ‘A Deep Learning-Based Intrusion Detection and Prevention System for Detecting and Preventing Denial-ofService Attacks’, IEEE Access, 10.
8. Krishna, A., Lal, A.M.A., Mathewkutty, A.J., et al. (2020) ‘Intrusion Detection and Prevention System Using Deep Learning’, Proc. Int. Conf. on Electronics and Sustainable Communication Systems (ICESC).
9. Lansky, J., Ali, S., Karim, S.H.T., et al. (2021) ‘Deep-Learning Based Intrusion Detection Systems: A Systematic Review’, IEEE Access, 9.

Appendix A: Screenshots


CyberID
Advanced Intrusion Detection System
Our cutting-edge system leverages machine learning algorithms (SVM & Random Forest) to accurately classify normal network activities from potential security threats. With sophisticated data preprocessing and feature scaling, we provide reliable intrusion detection to keep your network safe.
NORMAL
SCANNING...
Real-Time Detection
Our system continuously monitors network traffic patterns and identifies suspicious activities in real-time.
ML-Powered
Using Support Vector Machines and Random Forest algorithms to achieve high accuracy in threat detection.
Advanced Analytics
Comprehensive data preprocessing, scaling, and outlier management for reliable intrusion detection.


Upload Network Log File
Drag & drop your log file here or click to browse
Analyze Network Traffic
> System ready for intrusion detection analysis
> Upload a log file to begin...
CyberID © 2025 | Advanced Intrusion Detection System


> System ready for intrusion detection analysis
> Upload a log file to begin...
> File "x_test (1).csv" received...
> Processing log data...
> Applying feature scaling...
> Running ML prediction models (SVM, RF)...


CyberSecurity Dashboard
Upload Log File
Threats Detected: 3
Last Updated: Just Now
Server Status: Online
Analysis Results
Prediction: NORMAL
Service: http | Protocol: tcp | Duration: 0ms | Failed Logins: 0
Prediction: ABNORMAL
Service: ftp_data | Protocol: tcp | Duration: 491ms | Failed Logins: 0
Prediction: ABNORMAL
Service: telnet | Protocol: tcp | Duration: 100ms | Failed Logins: 5
- Failed login count exceeds threshold
- High remote error rate
Prediction: ABNORMAL
Attack Type Distribution
normal
abnormal
Failed Login Attempts
Failed Login %
60
50
40
30
20
10
0
abnormal
normal

Protocol Usage Statistics
Normal
Abnormal
120,000
100,000
80,000
60,000
40,000
20,000
0
icmp
tcp
udp
Service Distribution
Normal
Abnormal
http
private
domain_u
smtp
ftp_data
eco_i
other
ecr_i
telnet
finger
0
5,000
10,000
15,000
20,000
25,000
30,000
35,000
40,000
Recent Security Events
Multiple failed login attempts detected from IP 192.168.1.45
2025-04-19 14:32:15
Unusual SSH connection from unknown IP address 203.45.67.89
2025-04-19 14:25:03
Firewall rule update: Added block for IP range 45.67.89.0/24
2025-04-19 14:20:47
© 2025 Cybersecurity Dashboard | CyberID